Project Title

# Complete DevOps Pipeline (Git + Jenkins + Docker + AWS)

**A Project Report**

***In the partial fulfillment of the award of the degree of***

**B. Tech**

at


| Name of the students | Roll no |
|---|---|
| • Adarsh Kumar | 55 |
| • Gautam Rishi | 66 |
| • Ashutosh Yadav | 64 |
| • Raushan Kumar | 60 |



ARDENT COMPUTECH PVT. LTD.

# NARULA INSTITUTE OF TECHNOLOGY

**Ardent Computech Pvt. Ltd**


ARDENT COMPUTECH PVT. LTD.

## Certificate from the Mentor

This is to certify that Adarsh Kumar, Gautam Rishi, Ashutosh Yadav, Raushan Kumar has completed the project titled **Complete DevOps Pipeline (Git + Jenkins + Docker + AWS)** under my supervision during the period from **7th July** to **11th July** which is in partial fulfillment of requirements for the award of the **B.TECH** and submitted to Department **Data Science of Narula Institute of Technology.**

_____

*Signature of the Mentor*

**Date:**

## Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **Sourav Goswami** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Ardent Computech Pvt. Ltd.** for their support.

# TABLE OF CONTENTS

# CHAPTER 1

# INTRODUCTION

## 1.1 Project Overview

This project is titled “Deployment of a Static Portfolio Website on AWS EC2 Using Docker and Nginx.” The implementation combines two distinct areas of web engineering: the design of a personal portfolio website using standard front-end technologies, and the deployment of that website on a cloud server using containerization. In this project, the front-end was built first, using HTML for structure, CSS for styling, and JavaScript for interactivity, and only after the site was working correctly on a local machine was attention shifted to hosting it on the internet through Amazon Web Services (AWS).

The portfolio website itself is organized into six sections — Home, About, Skills, Projects, Resume, and Contact — each of which is intended to present a different aspect of the developer’s profile to a visitor. The Home section acts as a landing page, the About section introduces background information, the Skills section lists technical competencies, the Projects section highlights completed work, the Resume section provides access to a downloadable resume, and the Contact section allows visitors to reach out. The page is built to be responsive, so that it renders correctly on both desktop and mobile screen sizes.

Rather than relying on a traditional shared-hosting service or a static-site host such as GitHub Pages, this project deploys the website on a self-managed AWS EC2 instance. The reasoning behind this choice was to gain direct, hands-on exposure to cloud infrastructure, server administration, and containerization — skills that are increasingly expected of web developers in the industry. The implementation uses Git and GitHub for version control of the source code, Docker to package the website into a portable, self-contained image, and Nginx, running inside the Docker container, to serve the site’s static files over HTTP. The complete workflow, from writing the first line of HTML to viewing the deployed site through the EC2 instance’s public IP address, is documented in the chapters that follow.

In this project, the deployment process was approached as a sequence of well-defined stages — provisioning the cloud instance, securing it with appropriate firewall rules, installing the required software, containerizing the application, and finally running and verifying the container. Each of these stages is discussed in detail in Chapter 4, while the underlying design decisions are covered in Chapter 2.

## 1.2 Objectives of the Project

The objectives set out for this project were defined to cover both the front-end development effort and the cloud deployment effort, since the project treats these as equally important halves of the same task. The specific objectives are as follows:

- **Development of a Responsive Portfolio Website:** To design and build a personal portfolio site using HTML, CSS, and JavaScript that renders correctly across different screen sizes and devices.
- **Version Control and Source Management:** To maintain the complete source code of the website in a Git repository and host it on GitHub, so that changes can be tracked and the code can be pulled onto any server.
- **Provisioning of Cloud Infrastructure:** To create and configure an AWS EC2 instance running Ubuntu, which acts as the remote host for the deployed website.
- **Network Security Configuration:** To configure EC2 Security Groups so that only the required ports — port 22 for SSH access and port 80 for HTTP traffic — are exposed to the internet.
- **Containerization of the Application:** To package the website using Docker, writing a Dockerfile based on the official Nginx image, so that the site can be built and run consistently regardless of the underlying host configuration.
- **Deployment and Verification:** To build the Docker image on the EC2 instance, run it as a container, and confirm that the website is reachable and correctly rendered when accessed through the instance's public IP address.
- **Documentation of the Workflow:** To record each stage of the deployment process in sufficient detail that it could be reproduced by another developer following the same steps.

Taken together, these objectives were meant to ensure that the finished project would not simply be a website that happens to be reachable online, but a demonstration of a repeatable, containerized deployment pipeline that could, in principle, be extended to other static or lightly dynamic web applications in the future.

## 1.3 Scope of the Project

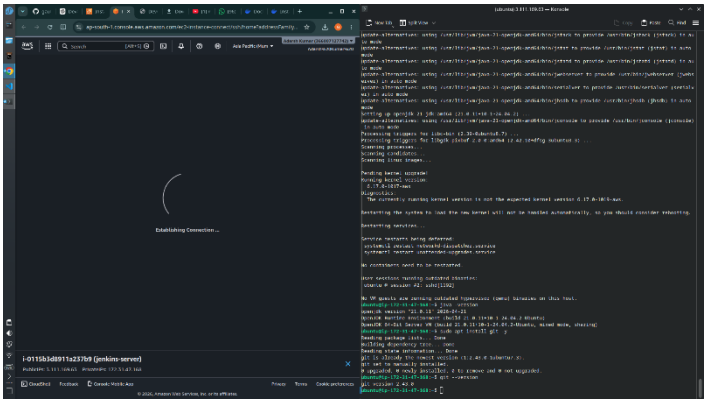
The scope of this project is limited to the design, development, and deployment of a single static portfolio website, and does not extend into areas such as backend application logic, databases, or user authentication, since the site itself is static in nature. The project covers the complete path from writing the website's front-end code to making it publicly accessible on the internet through a single AWS EC2 instance.

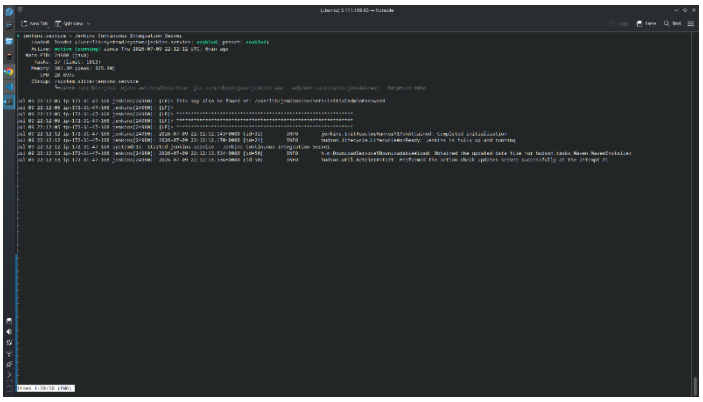
Within this scope, the project specifically includes:

- Design and development of the portfolio website front-end using HTML, CSS, and JavaScript.
- Version control of the project source code using Git, with the repository hosted on GitHub.
- Provisioning of a single Ubuntu-based EC2 instance on AWS.
- Configuration of EC2 Security Groups for SSH and HTTP access.
- Installation of Git and Docker on the EC2 instance.

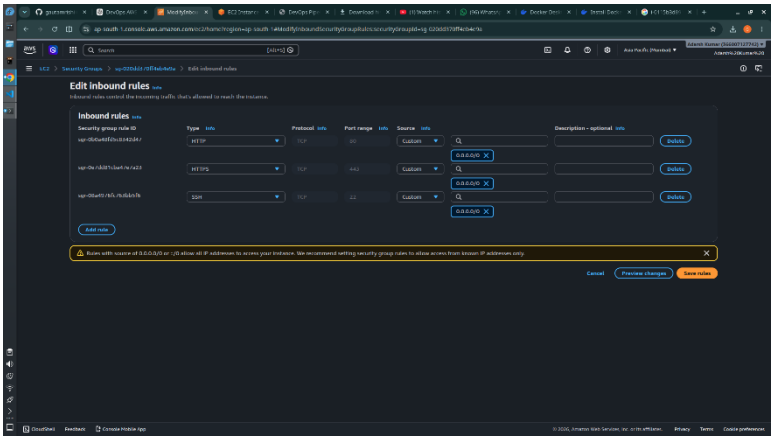
- Writing a Dockerfile based on the official Nginx image to serve the website.
- Building the Docker image and running it as a container on the EC2 instance.
- Verification of the deployed website by accessing it through the EC2 instance's public IP address.

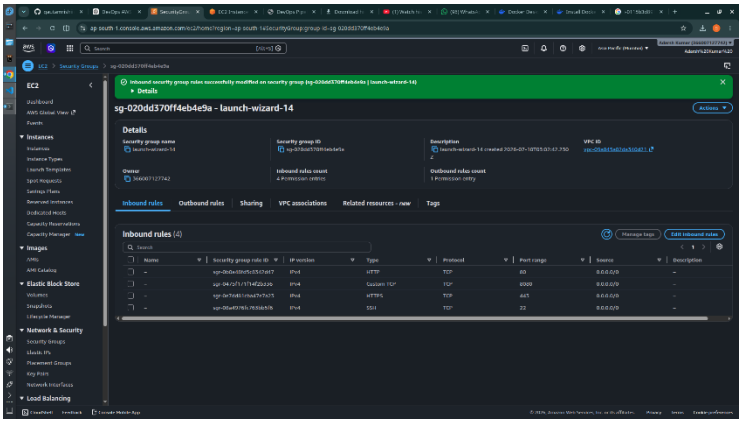
The project does not, in its current form, cover the use of a custom domain name, HTTPS/SSL certificate configuration, load balancing across multiple instances, container orchestration tools such as Kubernetes, or continuous integration/continuous deployment (CI/CD) pipelines for automated redeployment. These aspects are recognized as natural extensions of the work and are discussed briefly as possible future improvements in the concluding chapter, but they were kept outside the boundary of the present implementation so that the core deployment workflow could be completed and documented thoroughly





.



# CHAPTER 2

# SYSTEM DESIGN

## 2.1 Architecture

The overall architecture of this project is deliberately simple, since the application being deployed is a static website rather than a multi-tier system with a separate backend and database. Even so, the deployment path involves several distinct layers, each with a clear responsibility, and understanding how these layers connect is central to understanding the project as a whole.

At the top of the architecture is the local development environment, where the website's HTML, CSS, and JavaScript files were written and tested in a browser before anything was pushed to the cloud. Once the site was working locally, the source code was committed to a Git repository and pushed to GitHub, which then served as the single source of truth for the project. From GitHub, the code was pulled onto an AWS EC2 instance running Ubuntu, which acts as the remote server for the project. Rather than installing Nginx directly on the Ubuntu host, the project wraps the website and an Nginx server together inside a Docker image, and it is this image that is actually run as a container on the EC2 instance. The container listens on port 80, which the instance's Security Group exposes to the public internet, so that any visitor who requests the EC2 instance's public IP address is served the website by Nginx from inside the container.

This layered design keeps each concern separate: the browser-side code is only concerned with presentation, GitHub is only concerned with versioning, the EC2 instance is only concerned with providing compute and network access, and the Docker container is only concerned with running Nginx and serving the site's files. In this project, this separation made the deployment easier to reason about and easier to redo from scratch, since rebuilding the Docker image or relaunching the EC2 instance does not require touching the website's source code at all.

*[Insert Architecture Diagram Here]*

The diagram above traces the path of the project from the developer's local machine through GitHub and onto the AWS EC2 instance, showing the Docker container running inside the instance and Nginx serving the site out to an end user's browser over HTTP. The arrows in the diagram represent, in order, the pushing of code to GitHub, the pulling of that code onto the EC2 instance, the building of the Docker image from the project's Dockerfile, the running of that image as a container, and finally the flow of an HTTP request from a visitor's browser to the Nginx process inside the container and back.

## 2.2 Frontend Design

The frontend of the portfolio website was designed before any cloud infrastructure was touched, since the reasoning followed in this project was that the deployment pipeline should have a finished, working artifact to deploy rather than being built around an incomplete site. The frontend uses plain HTML, CSS, and JavaScript, without a framework, since the site does not require client-side routing, state management, or any of the complexity that a framework is meant to address.

The website is organized around a single index.html file that contains all six sections — Home, About, Skills, Projects, Resume, and Contact — as distinct, in-page segments that a visitor can reach either by scrolling or by clicking a corresponding link in the navigation bar. This single-page structure was chosen because it keeps the site fast to load, since the browser only ever has to fetch one HTML document, and it avoids the extra server configuration that would be needed to route between multiple HTML pages inside the Nginx container.

Styling is handled through a dedicated CSS file that defines the color scheme, typography, and layout of every section. Particular attention was paid to responsive design: the layout uses relative units and CSS Flexbox/Grid rules so that the page reflows correctly on narrower screens, and media queries adjust font sizes, spacing, and the navigation bar's appearance on mobile devices. JavaScript is used sparingly and only where it improves the user experience — for example, to enable smooth scrolling between sections, to toggle a mobile navigation menu, and to handle simple interactive elements such as a project filter or a contact form validation message.

The project's frontend files are organized into a small, predictable folder structure so that the same layout could later be copied into the Docker image without modification:

- index.html — the single HTML document containing all six sections of the site.
- css/ — a folder containing the stylesheet(s) that define the site's appearance.
- js/ — a folder containing the JavaScript file(s) that add interactivity to the page.
- assets/ — a folder containing images, icons, and the downloadable resume file used across the site.

*[Insert Figure 2.1 Here]*

**Figure 2.1: Folder Structure of the Portfolio Website Source Code**

Keeping the frontend self-contained in this way meant that, later in the project, the entire folder could be copied asis into the Docker build context without any changes to file paths or references, which considerably simplified the Dockerfile discussed in Section 2.4.

## 2.3 AWS Infrastructure

The cloud infrastructure for this project consists of a single Amazon EC2 instance, which was chosen over more elaborate AWS services such as Elastic Beanstalk or ECS specifically because the goal of the project was to understand and control every step of the deployment manually, rather than delegating it to a managed service.

The EC2 instance was launched using an Ubuntu Server image, since Ubuntu is widely documented, has broad package support, and is a common choice for hosting Docker workloads. The exact instance type, storage size, and AWS region used for this deployment are implementation-specific choices:

- EC2 Instance Type: [To be filled after implementation]
- AMI / Ubuntu Version: [To be filled after implementation]
- AWS Region: [To be filled after implementation]
- Storage Volume Size: [To be filled after implementation]

Alongside the instance itself, an AWS key pair was generated at launch time and used to authenticate SSH connections to the instance, and this key was downloaded and kept only on the local machine, since it is the only credential that allows administrative access to the server.

| Type | Protocol | Port | Source | Purpose |
|---|---|---|---|---|
| SSH | TCP | 22 | My IP / 0.0.0.0/0 | Allows secure shell access to configure instance |
| HTTP | TCP | 80 | 0.0.0.0/0 | Allows visitors' browsers to reach the Nginx web server |

**Security Group Configuration**

Network access to the EC2 instance is controlled through an AWS Security Group, which acts as a virtual firewall attached to the instance. For this project, the Security Group was configured with exactly two inbound rules, keeping the instance's exposed surface area as small as possible:

Restricting the Security Group to only these two ports means that no other service on the instance is reachable from the internet, which reduces the attack surface considerably compared with leaving all ports open. In this project, the SSH rule was used only during setup and configuration, while the HTTP rule remains open permanently so that the deployed website stays reachable by visitors.

*[Insert Figure 2.2 Here]*

**Figure 2.2: AWS EC2 Security Group Inbound Rules**

## 2.4 Docker Architecture

Docker is the layer that connects the static website described in Section 2.2 to the EC2 instance described in Section 2.3. Rather than installing and configuring Nginx directly on the Ubuntu host, this project packages the website

together with an Nginx server inside a Docker image, so that the entire runtime environment for the site is defined in a single, version-controlled Dockerfile.

The Docker image for this project is built starting from the official Nginx image available on Docker Hub, which already contains a working installation of the Nginx web server configured with sensible defaults. On top of this base image, the project's static files — the index.html file, the css/ and js/ folders, and the assets/ folder — are copied into Nginx's default web root directory inside the container, so that Nginx serves them automatically as soon as the container starts.

The Dockerfile used in this project therefore performs three essential actions: it selects the official Nginx image as the base, it copies the website's source files into the container's web root, and it exposes port 80 so that the container can accept incoming HTTP connections. Because the base image already ships with an entrypoint that starts Nginx in the foreground, no additional startup script is required.

```
FROM nginx:latest

COPY . /usr/share/nginx/html

EXPOSE 80
```

This Dockerfile keeps the image lightweight, since it adds nothing beyond the website's own files on top of the official Nginx base image, and it keeps the deployment reproducible, since building this Dockerfile on any machine — not just the specific EC2 instance used for this project — produces an identical container.

**Container Runtime Architecture**

Once built, the Docker image is run as a container on the EC2 instance. Docker maps a port on the host machine to port 80 inside the container, which is the port Nginx listens on. This means that requests arriving at the EC2 instance's public IP address on port 80 are forwarded by the operating system to the Docker engine, which in turn forwards them into the running container, where Nginx picks them up and serves the corresponding static file.

*[Insert Docker Architecture Diagram Here]*

The diagram illustrates how the Docker engine on the EC2 instance sits between the host operating system and the running container, and how a request on the host's port 80 is routed into the container's internal port 80, where the Nginx process serves the requested file from /usr/share/nginx/html. This isolation is what allows the website to be moved to a different server in the future simply by re-running the same Docker image, without repeating any of the manual Nginx configuration that a non-containerized deployment would require.

## 2.5 Deployment Workflow

The deployment of this project follows a fixed sequence of steps, each of which builds on the one before it. The workflow was kept intentionally linear so that it could be followed, reproduced, and documented without ambiguity:

1. An AWS EC2 instance running Ubuntu is created through the AWS Management Console.
2. The instance's Security Group is configured to allow inbound traffic on port 22 (SSH) and port 80 (HTTP), as described in Section 2.3.
3. A connection to the EC2 instance is established from the local machine using SSH and the downloaded key pair.
4. Git is installed on the EC2 instance using the Ubuntu package manager.
5. The project's GitHub repository is cloned onto the EC2 instance using Git.
6. Docker is installed on the EC2 instance, along with any dependencies it requires.
7. A Dockerfile based on the official Nginx image is written inside the cloned project directory, as described in Section 2.4.
8. The Docker image is built on the EC2 instance using the docker build command, which reads the Dockerfile and packages the website's files into an image.
9. The built image is run as a Docker container, with port 80 on the host mapped to port 80 inside the container.
10. The website is verified by opening a browser and navigating to the EC2 instance's public IP address, where the deployed site should now be visible.

*[Insert Deployment Workflow Diagram Here]*

The flowchart represents this ten-step sequence visually, showing how each stage — from provisioning the instance to final verification in the browser — depends on the successful completion of the stage before it. In this project, following the steps strictly in this order avoided several common pitfalls, such as attempting to install Docker before the Security Group rules were in place, or attempting to build the image before the repository had been cloned.

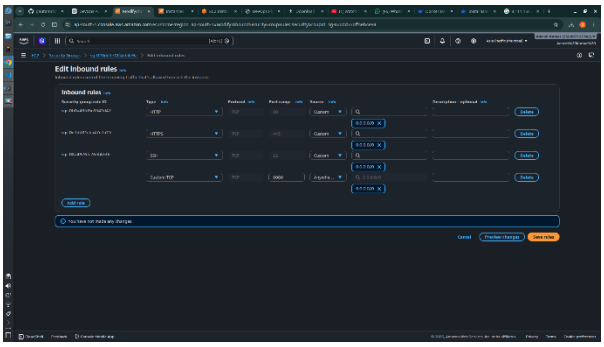
## 2.6 Components Used

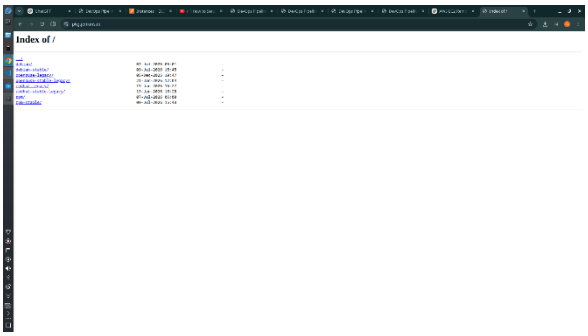
The table below summarizes the principal technologies and services used in the design and deployment of this project, along with the specific role each one plays in the overall system.

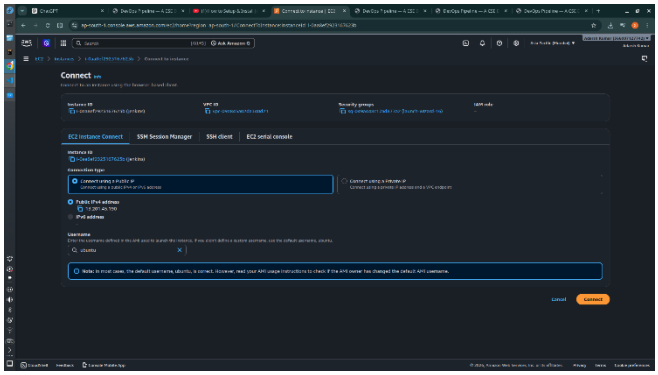
| Component | Category | Role in the Project |
|---|---|---|
| HTML5 | Frontend | Defines the structure and content of the portfolio website's sections. |
| CSS3 | Frontend | Provides styling, layout, and responsive design across screen sizes. |

| | | |
|---|---|---|
| JavaScript | Frontend | Adds interactivity such as smooth scrolling and navigation behaviour. |
| Git | Version Control | Tracks changes to the project's source code locally. |
| GitHub | Version Control | Hosts the remote repository that is cloned onto the EC2 instance. |
| AWS EC2 | Cloud Infrastructure | Provides the virtual server on which the website is deployed. |
| Ubuntu Server | Operating System | Runs on the EC2 instance and hosts the Docker engine. |
| AWS Security Groups | Network Security | Restricts inbound network access to ports 22 and 80 only. |
| Docker | Containerization | Packages the website and Nginx server into a portable image and runs it as a container. |
| Nginx | Web Server | Serves the static website files over HTTP from inside the Docker container. |
| SSH | Remote Access | Provides secure administrative access to the EC2 instance from the local machine. |

Each of these components was chosen for its simplicity and its wide use in real-world deployments, so that the skills developed while completing this project — configuring a cloud server, securing it, containerizing an application, and serving it through a reverse-proxy-capable web server — would carry over directly to other, more complex projects in the future.




# CHAPTER 3

# WEBSITE DEVELOPMENT

## 3.1 HTML Structure

The HTML for this project forms the skeleton on top of which the CSS styling and JavaScript behaviour described later in this chapter are layered. Since the site is a single-page portfolio, the entire markup lives inside one index.html file, with each of the six sections — Home, About, Skills, Projects, Resume, and Contact — marked out as its own <section> element carrying a unique id attribute. These ids are what allow the navigation bar to link directly to a section using an in-page anchor, and they are also what the JavaScript scroll-tracking logic in Section 3.3 refers back to.

At the top of the document, the <head> element declares the character encoding, a responsive viewport meta tag, the page title, and links to the external stylesheet and any icon fonts used for the site's social and skill icons. In this project, the viewport meta tag was treated as a non-negotiable requirement rather than an afterthought, since without it the responsive CSS rules described in Section 3.4 would have no effect on mobile devices.

The <body> begins with a fixed navigation bar containing the site's logo or name on the left and a list of anchor links — Home, About, Skills, Projects, Resume, and Contact — on the right. On narrower screens, this list is replaced by a hamburger icon that toggles a collapsible menu, which is discussed further in Section 3.3. Below the navigation bar, each section follows in document order:

- **Home:** Contains a short introductory heading, the developer's name and role, and a call-to-action button that scrolls the visitor down to the About section.
- **About:** Contains a short paragraph of biographical text alongside a profile image, introducing the developer's background and interests.
- **Skills:** Contains a grid of skill items, each represented by an icon and a label, grouped loosely by category such as languages, frameworks, and tools.
- **Projects:** Contains a grid of project cards, each with a title, a short description, a technology tag list, and links to the project's live demo and source code.
- **Resume:** Contains a short summary paragraph and a button that lets a visitor view or download the developer's resume as a PDF file.
- **Contact:** Contains a contact form with fields for name, email, and message, along with links to the developer's email address and social profiles.

A representative excerpt of the HTML skeleton, showing how the sections are laid out relative to one another, is given below:

```
<body>
  <nav class="navbar">
    <a href="#home" class="logo">Portfolio</a>
    <ul class="nav-links">
      <li><a href="#home">Home</a></li>
      <li><a href="#about">About</a></li>
      <li><a href="#skills">Skills</a></li>
      <li><a href="#projects">Projects</a></li>
      <li><a href="#resume">Resume</a></li>
      <li><a href="#contact">Contact</a></li>
</ul>
    <div class="hamburger" id="hamburger"></div>
  </nav>

  <section id="home">...</section>
  <section id="about">...</section>
  <section id="skills">...</section>
  <section id="projects">...</section>
  <section id="resume">...</section>
  <section id="contact">...</section>

  <footer>...</footer>
  <script src="js/script.js"></script> </body>
```

Semantic elements such as <nav>, <section>, and <footer> were used throughout in place of generic  tags wherever possible, since this keeps the markup readable, gives each part of the page a clear purpose, and improves how the page is interpreted by screen readers and search engines.

*[Insert Figure 3.1 Here]*

**Figure 3.1: Home Section of the Portfolio Website**

## 3.2 CSS Styling

The visual design of the portfolio is defined entirely in a single external stylesheet, css/style.css, which is linked from the HTML document's <head>. Keeping the CSS in one file, rather than splitting it across several, was a deliberate choice for a project of this size, since it avoids the overhead of managing multiple stylesheet imports for what is ultimately a small number of sections.

At the top of the stylesheet, a small set of CSS custom properties (variables) is defined on the :root selector, capturing the site's primary color, accent color, background color, and font family in one place. Referencing these variables throughout the rest of the stylesheet, rather than repeating raw color codes, made it straightforward to keep the color scheme consistent across every section and to adjust it later without having to search through the entire file.

```
:root {
  --color-primary: #2563eb;
  --color-bg: #ffffff;
  --color-text: #1f2937;
  --font-main: 'Segoe UI', sans-serif;
}  body {
margin: 0;
  font-family:        var(--font-main);
color: var(--color-text);
  background-color: var(--color-bg);
}
```

Layout within each section relies mainly on Flexbox and CSS Grid rather than older techniques such as floats. The navigation bar uses Flexbox to align the logo, the links, and the hamburger icon along a single row, while the Skills and Projects sections use CSS Grid to arrange their items into an evenly spaced, wrapping grid that adjusts the number of columns depending on the available width. This combination was chosen because Flexbox suits the onedimensional alignment needed for the navbar, while Grid suits the two-dimensional card layouts used elsewhere on the page.

Beyond layout, the stylesheet defines consistent spacing, rounded corners, and subtle box-shadows for the project and skill cards, along with hover transitions that slightly lift a card or change a link's color when the user's pointer moves over it. These small interactive touches were added to make the site feel more polished without introducing any JavaScript, since they are implemented purely through CSS transition and transform properties.

- A shared .container class constrains content to a maximum width and centers it horizontally on large screens.
- A .btn class defines a consistent button style, reused for the Home section's call-to-action and the Resume section's download button.
- A .card class defines the shared appearance of project and skill cards, including padding, border-radius, and box-shadow.
- Utility classes such as .text-center and .section-title keep heading alignment and spacing consistent across sections.

*[Insert Figure 3.2 Here]*

**Figure 3.2: Skills and Projects Sections Showing the Card-Based Layout**

## 3.3 JavaScript Functionality

JavaScript is used in this project only where it meaningfully improves the browsing experience, rather than as a default choice for every interaction. All of the site's interactive behaviour is contained in a single file, js/script.js, which is loaded at the end of the <body> so that it executes only after the HTML elements it refers to have already been parsed.

The first piece of functionality is the mobile navigation toggle. When the hamburger icon is clicked on a narrow screen, a JavaScript event listener adds or removes a CSS class on the navigation links container, sliding the menu in and out

of view. This keeps the show/hide animation itself defined in CSS, with JavaScript responsible only for toggling the class that triggers it.

```
const hamburger = document.getElementById('hamburger'); const
navLinks = document.querySelector('.nav-links');
 hamburger.addEventListener('click', () =>
{   navLinks.classList.toggle('active');
});
```

The second piece of functionality is smooth scrolling between sections. Rather than the browser jumping instantly to an anchor target, an event listener attached to each navigation link intercepts the default click behaviour and calls scrollIntoView with a smooth behaviour option, so that the page glides to the corresponding section.

```
document.querySelectorAll('.nav-links a').forEach(link => {
link.addEventListener('click', (e) => {
    e.preventDefault();
    const target = document.querySelector(link.getAttribute('href'));
target.scrollIntoView({ behavior: 'smooth' });
navLinks.classList.remove('active');
  });
});
```

A third piece of functionality highlights the navigation link corresponding to the section currently in view as the visitor scrolls down the page, using the Intersection Observer API to detect when a section crosses into the viewport and adding an .active class to the matching link. Finally, the contact form includes a lightweight client-side validation function that checks whether the name, email, and message fields are filled in and that the email field contains an '@' character before allowing submission, and displays an inline message if any field is missing.

```
const form = document.getElementById('contact-form');
 form.addEventListener('submit', (e) =>
{
  e.preventDefault();
  const name = document.getElementById('name').value.trim();
const email = document.getElementById('email').value.trim();
const message = document.getElementById('message').value.trim();
   if (!name || !email.includes('@') || !message) {
document.getElementById('form-status').innerText =
      'Please fill in all fields correctly.';
return;
  }   document.getElementById('form-
status').innerText =
    'Message sent successfully!';

  form.reset(); });
```

None of these scripts depend on an external library or framework; they rely entirely on standard DOM APIs available in modern browsers. This was a deliberate decision in this project, since pulling in a framework such as React or a library such as jQuery would have added build tooling and complexity disproportionate to the size of the site, and would also have complicated the Docker image described in Chapter 2, which simply copies static files into the Nginx web root without any build or compilation step.

*[Insert Figure 3.3 Here]*

**Figure 3.3: Contact Section Showing the Form Validation Message**

## 3.4 Responsive Design

Responsive design was treated as a core requirement of the frontend rather than a late addition, since the website needed to present well on both desktop monitors and mobile phone screens. The approach followed in this project combines a fluid, relative-unit-based layout with a small number of CSS media queries that adjust specific properties at defined breakpoints.

Wherever possible, widths are expressed using percentages, rem units, or CSS Grid's auto-fit and minmax functions instead of fixed pixel values, so that elements naturally resize as the viewport changes. The Skills and Projects grids, for example, use a repeat(auto-fit, minmax(220px, 1fr)) column definition, which allows the browser to automatically fit as many columns as will comfortably hold a 220-pixel-wide card and to reflow them onto fewer columns as the screen narrows, without needing a separate media query for every intermediate screen width.

For layout changes that fluid sizing alone cannot handle — such as collapsing the navigation bar into a hamburger menu, or stacking the About section's image and text vertically instead of side by side — the stylesheet defines media queries at two main breakpoints, targeting tablet-sized and mobile-sized screens:

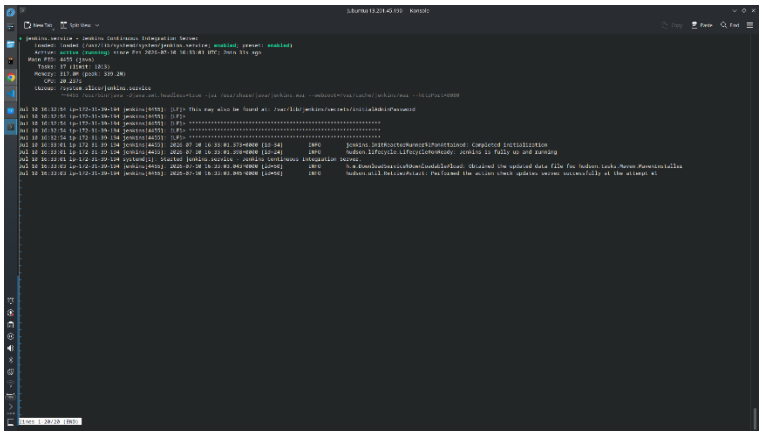
```
@media (max-width: 768px) {
  .nav-links {     display:
none;     flex-direction:
column;     position:
absolute;     top: 60px;
right: 0;
  }
  .nav-links.active { display: flex; }
  .hamburger { display: block; }
}

@media (max-width: 480px) {
  .about-content { flex-direction: column; text-align: center; }
  .section-title { font-size: 1.5rem; }
}
```

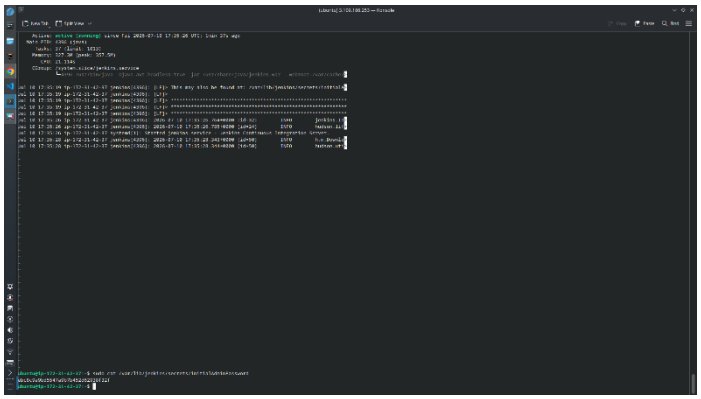
Images across the site, including the profile picture in the About section and any project screenshots in the Projects section, are constrained with a max-width of 100% and a height of auto in CSS, so that they scale down proportionally on smaller screens rather than overflowing their containers or forcing horizontal scrolling. Font sizes for headings are also reduced slightly at the narrower breakpoint, since headings sized for a desktop screen tend to wrap awkwardly on a phone-sized viewport.

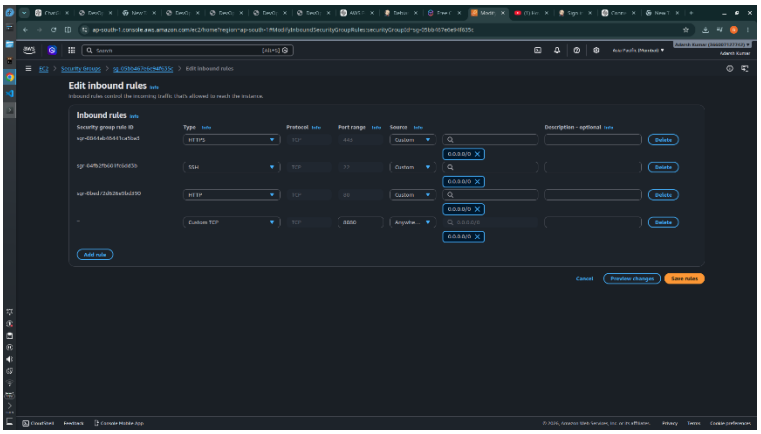
Testing of the responsive behaviour was carried out using the device toolbar in the browser's developer tools, checking the layout at common breakpoints corresponding to typical desktop, tablet, and mobile phone widths, in addition to resizing the browser window freely to check for any layout breakage between the defined breakpoints.
The results of this responsive testing are discussed further in Chapter 5.

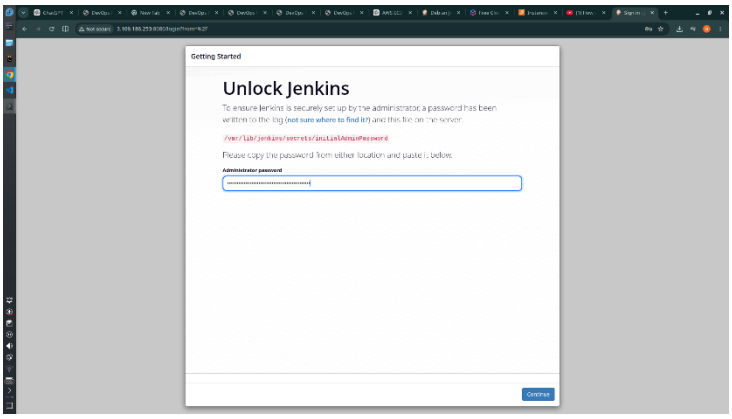
*[Insert Figure 3.4 Here]*

**Figure 3.4: Portfolio Website Displayed on a Mobile Screen Width**







# CHAPTER 4

# DEPLOYMENT ON AWS

This chapter walks through the deployment of the portfolio website in the exact order it was carried out, from pushing the finished frontend to GitHub through to viewing the live site in a browser. Each section corresponds to one stage of the ten-step workflow introduced in Section 2.5, and reproduces the commands used at that stage along with an explanation of what each command does and why it was needed.

## 4.1 GitHub Repository

Before any cloud resources were touched, the completed frontend — the index.html file along with the css/, js/, and assets/ folders described in Chapter 3 — was placed under version control using Git and pushed to a repository on GitHub. Keeping the code on GitHub, rather than only on the local machine, was essential to this project's workflow, since the EC2 instance later retrieves the code by cloning this repository directly rather than by any manual file transfer.

A local Git repository was first initialized inside the project folder, after which a .gitignore file was added to exclude editor-specific files and any local configuration that should not be tracked. The project was then committed and pushed to a new, empty repository created on GitHub:

```
git init git add . git commit -m "Initial commit:
portfolio website" git branch -M main
git remote add origin https://github.com/<username>/<repo-name>.git git
push -u origin main
```

Once pushed, the repository's GitHub page shows the complete folder structure of the project, matching what was described in Section 2.2, and this page also provides the HTTPS clone URL that is used later in Section 4.6 when the repository is cloned onto the EC2 instance.

*[Insert Figure 4.1 Here]*

**Figure 4.1: GitHub Repository Showing the Uploaded Project Files**

## 4.2 Launching AWS EC2 Instance

With the source code safely hosted on GitHub, the next stage was to provision the virtual server on which the website would eventually run. This was done through the AWS Management Console's EC2 dashboard, using the "Launch Instance" wizard rather than the AWS CLI, since the console's guided steps make each configuration choice — the AMI, the instance type, the key pair, and the storage volume — explicit and easy to review before the instance is created.

The launch wizard was stepped through as follows:

- **Name and Tags:** The instance was given a descriptive name so that it could be identified easily in the EC2 dashboard alongside any other instances in the account.
- **Application and OS Images (AMI):** An Ubuntu Server image was selected as the operating system for the instance, consistent with the design decision described in Section 2.3.
- **Instance Type:** An instance type was selected based on the compute and memory requirements of serving a static website through Nginx, which are modest. [To be filled after implementation]
- **Key Pair:** A new key pair was created and its private key file (.pem) was downloaded to the local machine, since this file is the only means of authenticating an SSH connection to the instance and cannot be redownloaded later.
- **Network Settings:** The Security Group described in Section 4.3 was configured at this stage, allowing SSH and HTTP traffic.
- **Configure Storage:** The default storage volume proposed by the wizard was retained, since a static website and a small Docker image do not require additional disk space beyond the default allocation.

After reviewing the summary panel, the instance was launched, and the EC2 dashboard was used to confirm that the instance transitioned from a pending state to a running state, at which point AWS assigned it a public IPv4 address that is used throughout the remainder of this chapter to reach the instance.

*[Insert Figure 4.2 Here]*

**Figure 4.2: AWS EC2 Instance Shown in the Running State on the Dashboard**

## 4.3 Configuring Security Groups

As introduced in Section 2.3, network access to the EC2 instance is restricted through a Security Group acting as a virtual firewall. This Security Group was configured during the instance launch wizard in Section 4.2, and its rules were subsequently reviewed and confirmed from the EC2 dashboard's "Security Groups" panel to ensure that exactly the required ports were open and nothing else:

Limiting the SSH rule's source to the developer's own IP address, rather than leaving it open to 0.0.0.0/0, was an additional precaution taken in this project so that only the machine actually being used for setup could attempt to authenticate over SSH, while the HTTP rule was deliberately left open to all sources since the website is meant to be publicly reachable.

| Type | Protocol | Port Range | Source | Purpose |
|---|---|---|---|---|
| SSH | TCP | 22 | My IP | Restricts administrative access to the instanc developer's own IP address |
| HTTP | TCP | 80 | 0.0.0.0/0 | Allows any visitor's browser to reach the web served by Nginx |

*[Insert Figure 4.3 Here]*

**Figure 4.3: Inbound Rules of the EC2 Instance's Security Group**

## 4.4 Connecting via SSH

Once the instance was running and its Security Group confirmed, the next step was to establish a remote terminal session to the instance from the local machine. This was done using the downloaded .pem key pair and the instance's public IP address, following the connection command provided by the EC2 dashboard's "Connect" panel.

Before the key could be used, its file permissions were restricted, since SSH refuses to use a private key that is readable by other users on the system:

```
chmod 400 my-key-pair.pem ssh -i "my-key-
pair.pem" ubuntu@<ec2-public-ip>
```

On the first connection, SSH displays the remote host's fingerprint and asks for confirmation before proceeding, after which a shell prompt on the Ubuntu instance appears, indicating that the connection was successful. From this point onward, every command described in the remaining sections of this chapter was executed inside this SSH session, directly on the EC2 instance.

*[Insert Figure 4.4 Here]*

**Figure 4.4: Terminal Session Connected to the EC2 Instance via SSH**

## 4.5 Installing Git

Although the Ubuntu AMI used for this instance comes with a minimal base set of packages, Git is not installed by default and had to be added before the project repository could be cloned. The instance's package index was refreshed first, to ensure that the version of Git installed was the latest one available in Ubuntu's repositories, after which Git itself was installed:

```
sudo apt update sudo
apt install git -y git
--version
```

The git --version command was used simply as a sanity check, confirming that the installation had completed successfully and that the git command was available on the instance's PATH before moving on to cloning the repository.

*[Insert Figure 4.5 Here]*

**Figure 4.5: Git Version Output Confirming Successful Installation**

## 4.6 Cloning the GitHub Repository

With Git installed, the project's source code was pulled onto the EC2 instance directly from the GitHub repository set up in Section 4.1, using the repository's HTTPS clone URL. This step brings the exact same index.html, css/, js/, and

assets/ files that were developed and tested locally in Chapter 3 onto the remote server, without any manual file copying:

```
git clone https://github.com/<username>/<repo-name>.git cd <repo-name>
ls
```

The ls command afterward was used to confirm that the cloned folder matched the expected project structure, since any mismatch at this stage — such as a missing folder or an incorrectly named file — would otherwise only surface later, and less clearly, when the Docker image was built in Section 4.9.

*[Insert Figure 4.6 Here]*

**Figure 4.6: Project Files Listed After Cloning the Repository onto the EC2 Instance**

## 4.7 Installing Docker

Docker does not ship with the base Ubuntu AMI either, so it was installed next, before the Dockerfile described in the following section could be built into an image. For this project, Docker was installed using Ubuntu's own package repository, which provides a straightforward installation path suited to a single-container deployment of this scale:

```
sudo apt update
sudo apt install docker.io -y sudo systemctl enable docker
sudo systemctl start docker sudo usermod -aG docker ubuntu
docker --version
```

The systemctl enable command ensures that the Docker service starts automatically if the instance is ever rebooted, while systemctl start brings the service up immediately for the current session. Adding the ubuntu user to the docker group allows Docker commands to be run without prefixing every one of them with sudo, after the SSH session is reopened; until then, sudo was used in front of the docker commands shown in the remaining sections.

*[Insert Figure 4.7 Here]*

**Figure 4.7: Docker Version Output Confirming Successful Installation**

## 4.8 Creating the Dockerfile

With Docker installed and the project files already present on the instance from Section 4.6, a Dockerfile was created inside the cloned repository's root directory to describe how the website should be packaged into an image. This is the same Dockerfile design discussed in Section 2.4, created here directly on the EC2 instance using a terminal text editor:

```
nano Dockerfile
```

The following content was entered into the Dockerfile and saved:

```
FROM nginx:latest

COPY . /usr/share/nginx/html
```

```
EXPOSE 80
```

This Dockerfile instructs Docker to start from the official Nginx image, copy every file in the current build context — which, since the Dockerfile sits at the root of the cloned repository, means the entire portfolio project — into Nginx's default web root, and declare that the resulting container listens on port 80. No further configuration files were required, since the official Nginx image's default configuration is already set up to serve static files from that directory.

*[Insert Figure 4.8 Here]*

**Figure 4.8: Dockerfile Contents Viewed on the EC2 Instance**

## 4.9 Building the Docker Image

Once the Dockerfile was in place, it was used to build a Docker image containing the website and the Nginx server together. The build command was run from inside the project directory, so that the Dockerfile's COPY instruction had access to all of the website's files as its build context:

```
sudo docker build -t portfolio-website .
```

Docker works through the Dockerfile's instructions one at a time, first pulling the official nginx:latest image from Docker Hub if it is not already cached locally, and then copying the project's files into a new image layer on top of it. The -t flag attaches a readable name, portfolio-website, to the resulting image so that it can be referred to easily in the next step, rather than by its automatically generated image ID. Once the build finished, the image's presence was confirmed by listing all locally available Docker images:

```
sudo docker images
```

*[Insert Figure 4.9 Here]*

**Figure 4.9: Docker Build Output Showing the Successfully Built Image**

## 4.10 Running the Docker Container

With the image built, it was run as a container, which is the point at which Nginx actually begins serving the website. The container was started in detached mode, so that it continues running in the background after the terminal session returns to the shell prompt, and port 80 on the EC2 instance was mapped to port 80 inside the container, since that is the port Nginx listens on by default:

```
sudo docker run -d -p 80:80 --name portfolio-container portfolio-website
```

The --name flag gives the running container a memorable name, portfolio-container, which makes it easier to stop, restart, or inspect the container later without needing to look up its automatically generated container ID. Once started, the container's status was verified using:

```
sudo docker ps
```

This command lists all currently running containers along with their port mappings, and was used to confirm that portfolio-container was running and that the 0.0.0.0:80->80/tcp port mapping was in place exactly as intended, before attempting to reach the site from a browser.

*[Insert Figure 4.10 Here]*

**Figure 4.10: Output of docker ps Showing the Running Portfolio Container**

## 4.11 Accessing the Website

With the container confirmed to be running and listening on port 80, and with the instance's Security Group already permitting inbound HTTP traffic as configured in Section 4.3, the final step was simply to open a web browser and navigate to the EC2 instance's public IP address:

```
http://<ec2-public-ip>
```

Loading this address displayed the portfolio website exactly as it had appeared during local development in Chapter 3, confirming that every stage of the pipeline — from the GitHub repository, through the EC2 instance and its Security Group, to the Docker container and the Nginx process running inside it — was working correctly end to end. This moment marked the completion of the core deployment workflow described in Section 2.5, with the site now publicly reachable by anyone with the instance's public IP address.

Because the public IP address assigned to an EC2 instance can change if the instance is stopped and restarted, this address was noted down for reference during testing, and the possibility of attaching a static Elastic IP address or a custom domain name was left as a future improvement, consistent with the boundaries of scope set out in Section 1.3.

*[Insert Figure 4.11 Here]*

**Figure 4.11: Portfolio Website Successfully Loaded via the EC2 Public IP Address**

# CHAPTER 5

# TESTING AND VALIDATION

Testing for this project was carried out incrementally rather than as a single activity performed only after the website was deployed. Since the project has two distinct halves — the frontend described in Chapter 3 and the deployment pipeline described in Chapter 4 — each half was validated on its own before being tested together as a complete system. This chapter documents the approach taken and the specific checks performed at each stage, along with the issues encountered and how they were resolved.

## 5.1 Testing Approach

The overall testing strategy followed in this project was to validate the site at three separate checkpoints, so that a fault could be traced back to the specific stage that introduced it rather than being discovered only once the entire pipeline had run end to end:

- **Local Testing:** The website was opened directly in a browser from the local file system before any code was committed, to confirm that the HTML, CSS, and JavaScript described in Chapter 3 behaved correctly in isolation.
- **Post-Deployment Container Testing:** After the Docker container was running on the EC2 instance, the container itself was checked directly on the instance — using commands such as docker ps and curl — before involving an external browser, so that any failure could be narrowed down to either the container or the network path.
- **End-to-End Browser Testing:** Finally, the site was loaded in a browser using the EC2 instance's public IP address, exactly as an external visitor would access it, to confirm that the Security Group, Docker container, and Nginx server were all working together correctly.

This staged approach meant that, in this project, a problem occurring after deployment could usually be isolated quickly: if the container-level test with curl succeeded but the browser test did not, the issue was almost certainly in the Security Group rather than in Docker or Nginx, and if the curl test itself failed, the issue was narrowed down to the container or the Dockerfile instead.

## 5.2 Functional Testing of the Website

Before any deployment activity began, the portfolio website was functionally tested locally to confirm that every interactive element described in Chapter 3 behaved as intended. This testing was carried out manually, by exercising each feature of the site directly in the browser and observing whether the result matched what was expected.

| Test Case | Steps | Expected Result | Status |
|---|---|---|---|
| Navigation links | Click each link in the navbar (Home, About, Skills, Projects, Resume, Contact) | Page scrolls smoothly to the corresponding section | Pass |
| Mobile menu toggle | Resize browser below 768px and click the hamburger icon | Navigation menu opens and closes correctly | Pass |
| Active link highlighting | Scroll manually through all sections | Corresponding navbar link is highlighted as each section comes into view | Pass |
| Resume download button | Click the download/view button in the Resume section | Resume PDF opens or downloads correctly | Pass |
| Contact form — empty fields | Submit the contact form with one or more fields left blank | Validation message is displayed and the form is not submitted | Pass |

| | | | |
|---|---|---|---|
| Contact form — invalid email | Enter a value without an '@' symbol in the email field and submit | Validation message is displayed | Pass |
| Contact form — valid input | Fill all fields correctly and submit | Success message is displayed and the form resets | Pass |

Each of these checks was repeated after every significant change to the HTML, CSS, or JavaScript files, so that a regression introduced by a later edit — for instance, a CSS rule that accidentally broke the mobile menu — would be caught immediately rather than being discovered only after the site had already been deployed.

*[Insert Figure 5.1 Here]*

**Figure 5.1: Local Functional Testing of the Contact Form Validation**

## 5.3 Deployment Verification Testing

Once the website was deployed following the steps in Chapter 4, a separate round of testing was carried out to confirm that the deployment pipeline itself — the EC2 instance, the Security Group, Docker, and Nginx — was functioning correctly, independently of the website's own frontend behaviour.

The first check was performed directly on the EC2 instance, over the existing SSH session, using curl to request the site from the container without involving the public internet or the Security Group at all:

```
curl -I http://localhost:80
```

A successful result here returns an HTTP 200 OK response header, confirming that Nginx inside the container was serving the website correctly on port 80, regardless of whether the site was reachable from outside the instance yet.

The container's logs were also inspected as a secondary check, to confirm that Nginx had started without errors:

```
sudo docker logs portfolio-container
```

With the container confirmed to be serving the site locally on the instance, the same request was then repeated from the local development machine, targeting the instance's public IP address instead of localhost, to confirm that the Security Group's HTTP rule was correctly forwarding external traffic through to the container:

```
curl -I http://<ec2-public-ip>:80
```

Passing this second check confirmed that the full chain — Security Group, EC2 networking, Docker's port mapping, and Nginx — was working together correctly, which was then further confirmed visually by loading the same address in a browser, as already shown in Figure 4.11.

| Test Case | Command / Action | Expected Result | Status |
|---|---|---|---|
| Container running | sudo docker ps | portfolio-container listed with status 'Up' | Pass |
| Local container response | curl -I http://localhost:80 (on instance) | HTTP/1.1 200 OK returned | Pass |
| External response | curl -I http://<public-ip>:80 (from local machine) | HTTP/1.1 200 OK returned | Pass |

| | | | |
|---|---|---|---|
| Browser access | Navigate to http://<public-ip> in a browser | Website loads and renders correctly | Pass |
| Security Group restriction | Attempt to reach a non-permitted port, e.g. port 8080 | Connection times out / is refused | Pass |

*[Insert Figure 5.2 Here]*

**Figure 5.2: curl Output Confirming an HTTP 200 OK Response from the Deployed Container**

## 5.4 Cross-Browser and Responsive Testing

Since the website is meant to be viewed by visitors on a range of devices and browsers, it was tested across the major desktop browsers as well as at several simulated screen widths, using the responsive design rules described in Section 3.4.

- Google Chrome — used as the primary development and testing browser throughout the project.
- Mozilla Firefox — checked to confirm that Flexbox, Grid, and the smooth-scroll behaviour rendered consistently.
- Microsoft Edge — checked as a Chromium-based browser to rule out any Chrome-specific quirks.

For responsive testing, the browser's built-in device toolbar was used to simulate common desktop, tablet, and mobile phone screen widths, checking specifically that the navigation bar collapsed into the hamburger menu at the 768px breakpoint defined in Section 3.4, that the Skills and Projects grids reflowed into fewer columns as the width decreased, and that no text or image overflowed its container at any tested width.

| Device / Viewport | Width Tested | Observed Behaviour | Status |
|---|---|---|---|
| Desktop | 1920px / 1366px | Full navigation bar and multi-column grids displayed | Pass |
| Tablet | 768px | Navigation collapses to hamburger menu; grids reflow to fewer columns | Pass |
| Mobile phone | 375px / 414px | Single-column layout; About section stacks vertically | Pass |
| Browser window resize | Freely resized between breakpoints | Layout reflows smoothly with no visible breakage | Pass |

This round of testing did not reveal any browser-specific inconsistencies, which was expected given that the site avoids browser-specific or experimental CSS and JavaScript features, relying instead on well-supported standards such as Flexbox, Grid, and the Intersection Observer API.

*[Insert Figure 5.3 Here]*

**Figure 5.3: Website Viewed at a Mobile Breakpoint Using the Browser's Device Toolbar**

## 5.5 Observations and Issues Encountered

A small number of issues were encountered while testing the deployment pipeline, and documenting them here is intended to help anyone reproducing this project avoid the same delays.

- **SSH Connection Refused:** On the first connection attempt, the SSH client returned a connection-refused error. This was traced back to the Security Group's inbound rule for port 22 not yet having propagated, and was resolved simply by waiting briefly and retrying the connection.
- **Permission Denied (publickey):** An early SSH attempt failed with a publickey permission error, which was caused by the .pem key file's permissions not having been restricted with chmod 400 beforehand, as described in Section 4.4.
- **Website Not Reachable Despite Running Container:** In one instance, docker ps confirmed the container was running, yet the site was not reachable from a browser. Checking the Security Group revealed that the HTTP rule for port 80 had been left out when the instance was first launched; adding the rule resolved the issue immediately, without needing to touch Docker or Nginx at all.
- **Docker Permission Errors:** Immediately after installing Docker and adding the ubuntu user to the docker group, docker commands still required sudo. This was because group membership changes only take effect in a new login session, and was resolved by disconnecting and reconnecting the SSH session.

None of these issues required any change to the website's frontend code or to the Dockerfile itself; each one was resolved at the infrastructure or configuration level, which reinforced one of the observations already made in Section 2.1 — that keeping the frontend, the container, and the cloud infrastructure as separate, independently testable layers made it straightforward to isolate and fix problems without one layer's troubleshooting spilling into another.

Taken together, the testing carried out in this chapter confirmed that the portfolio website functions correctly on its own, that the Docker container built from the Dockerfile in Section 2.4 serves the site correctly, and that the AWS infrastructure — the EC2 instance and its Security Group — correctly exposes that container to the public internet. This gives confidence that the deployment described in Chapter 4 is both functionally correct and reproducible.

# CHAPTER 6

# SNAPSHOTS

This chapter brings together, in one place, the visual evidence of each stage of the project described in the preceding chapters. Rather than repeating the explanations already given in Chapters 2 through 5, each section here provides a short reminder of what that stage involved and reserves space for the corresponding screenshot, so that the finished report can be read end to end as a visual record of the completed work alongside its written description. The screenshots themselves are to be captured from the actual implementation and inserted into the marked placeholders below.

## 6.1 Portfolio Website Screenshots

The portfolio website is the frontend artifact at the center of this project, built using HTML, CSS, and JavaScript as described in Chapter 3. It presents six sections — Home, About, Skills, Projects, Resume, and Contact — through

which a visitor can learn about the developer's background, technical skills, and completed work. Before being deployed, the site was tested locally in a browser to confirm that every section rendered and behaved as intended, as discussed in Section 5.2.

*[Insert Figure 6.1 Here]*

**Figure 6.1: Portfolio Website Displaying the Home Section**

This screenshot demonstrates the finished visual appearance of the portfolio website as seen by a visitor. It confirms that the layout, styling, and content described in Chapter 3 render correctly in a real browser rather than only in the underlying source code.

## 6.2 GitHub Repository

The project's source code is version-controlled using Git and hosted on GitHub, as described in Section 4.1. The repository holds the complete frontend — the index.html file along with the css/, js/, and assets/ folders — and serves as the single source that is later cloned onto the AWS EC2 instance. Maintaining the code on GitHub rather than transferring files manually was central to keeping the deployment process reproducible.

*[Insert Figure 6.2 Here]*

**Figure 6.2: GitHub Repository Page Showing the Project Files**

This screenshot demonstrates that the project's source code was successfully pushed to GitHub and that its folder structure matches the design described in Section 2.2. It provides visual confirmation that version control was in place before any cloud deployment activity began.

## 6.3 AWS EC2 Instance

The AWS EC2 instance is the virtual server on which the portfolio website is ultimately hosted, provisioned through the AWS Management Console as described in Section 4.2. The instance runs Ubuntu Server and, once launched, is assigned a public IP address that visitors use to reach the deployed website. Confirming that the instance reached a running state was the first infrastructure milestone in the deployment workflow.

*[Insert Figure 6.3 Here]*

**Figure 6.3: AWS EC2 Dashboard Showing the Instance in a Running State**

This screenshot demonstrates that the EC2 instance was successfully launched and is active, along with the public IP address later used to access the site. It confirms that the cloud infrastructure described in Section 2.3 was provisioned correctly before any software was installed on it.

## 6.4 Security Group Configuration

The Security Group attached to the EC2 instance controls which network ports are reachable from outside AWS, and was configured to allow only SSH (port 22) and HTTP (port 80) traffic, as described in Sections 2.3 and 4.3.

This configuration is what permits the developer to administer the instance remotely while also allowing visitors' browsers to reach the deployed website. Restricting access to only these two ports was an intentional security measure taken in this project.

*[Insert Figure 6.4 Here]*

**Figure 6.4: Inbound Rules of the EC2 Security Group**

This screenshot demonstrates that the Security Group's inbound rules were configured exactly as intended, exposing only ports 22 and 80. It provides evidence that the instance's network exposure was deliberately restricted rather than left open by default.

## 6.5 SSH Connection

Administrative access to the EC2 instance is carried out over SSH, using the private key generated at launch time, as described in Section 4.4. This connection is what allows commands — such as those used to install Git and Docker — to be executed directly on the remote instance from the local machine. Successfully establishing this connection was a prerequisite for every subsequent step of the deployment.

*[Insert Figure 6.5 Here]*

**Figure 6.5: Terminal Showing an Active SSH Session with the EC2 Instance**

This screenshot demonstrates that a secure shell connection to the EC2 instance was successfully established from the local machine. It confirms that the key pair and Security Group rule described in Sections 4.2 and 4.3 were correctly configured to permit remote access.

## 6.6 Git Installation

Git was installed on the EC2 instance so that the project's GitHub repository could later be cloned onto the server, as described in Section 4.5. Since the base Ubuntu image does not include Git by default, this installation step was a necessary prerequisite before the repository could be retrieved. The installation was verified using the git --version command.

*[Insert Figure 6.6 Here]*

**Figure 6.6: Terminal Output Confirming Git Installation on the EC2 Instance**

This screenshot demonstrates that Git was installed successfully on the instance and that the correct version is available on the command line. It confirms that the instance was ready for the repository cloning step described in Section 4.6.

## 6.7 Docker Installation

Docker was installed on the EC2 instance to provide the containerization runtime needed to build and run the website's image, as described in Section 4.7. This installation includes enabling and starting the Docker service so that it remains

active even after the instance is rebooted. The installation was verified using the docker --version command, in the same manner as the Git installation in Section 6.6.

*[Insert Figure 6.7 Here]*

**Figure 6.7: Terminal Output Confirming Docker Installation on the EC2 Instance**

This screenshot demonstrates that Docker was installed and is available on the instance, confirming that the environment was ready to build and run the project's Docker image. It provides a checkpoint between the infrastructure setup covered earlier and the containerization steps that follow.

## 6.8 Dockerfile

The Dockerfile defines how the portfolio website is packaged together with an Nginx server into a single Docker image, as described in Sections 2.4 and 4.8. It is a short, plain-text file consisting of three instructions — selecting the base Nginx image, copying the website's files into the container, and exposing port 80. This file was created directly on the EC2 instance inside the cloned project directory.

*[Insert Figure 6.8 Here]*

**Figure 6.8: Dockerfile Contents Displayed in the Terminal**

This screenshot demonstrates the exact contents of the Dockerfile as it exists on the EC2 instance, matching the design described in Section 2.4. It confirms that the file was created correctly before being used to build the Docker image in the next section.

## 6.9 Docker Image Build

Building the Docker image is the step at which the Dockerfile's instructions are executed, producing a selfcontained image that bundles the website's files with the official Nginx image, as described in Section 4.9. This step was carried out using the docker build command and tagged with a readable name, portfolio-website, for easy reference in later commands. A successful build output confirms that every instruction in the Dockerfile executed without error.

*[Insert Figure 6.9 Here]*

**Figure 6.9: Terminal Output Showing a Successful Docker Image Build**

This screenshot demonstrates that the Docker image was built successfully from the Dockerfile, with each build step completing without error. It confirms that the image was ready to be run as a container, as covered in the following section.

## 6.10 Running Docker Container

Running the built image as a container is the step at which Nginx actually begins serving the portfolio website, as described in Section 4.10. The container was started in detached mode with port 80 on the EC2 instance mapped to

port 80 inside the container, and was given the readable name portfolio-container. Confirming that the container was active and correctly mapped was the last check performed before attempting to reach the site externally.

*[Insert Figure 6.10 Here]*

**Figure 6.10: Terminal Output of docker ps Showing the Running Container**

This screenshot demonstrates that the Docker container was running and that its port mapping matched what was intended, exposing the website on port 80. It provides the final infrastructure-level confirmation before the website was verified in a browser.

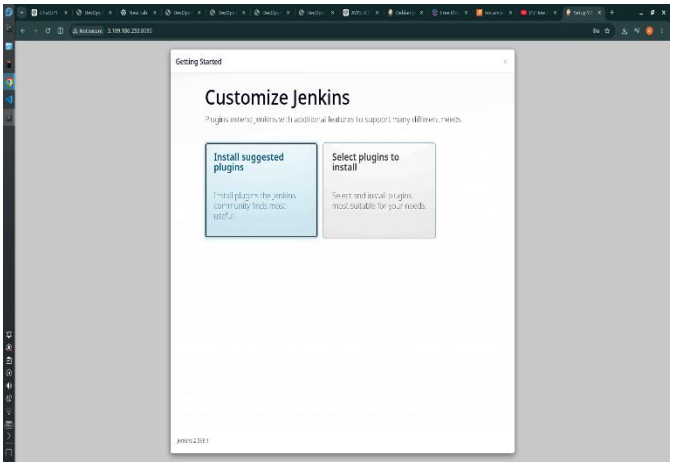
## 6.11 Final Website Running on EC2

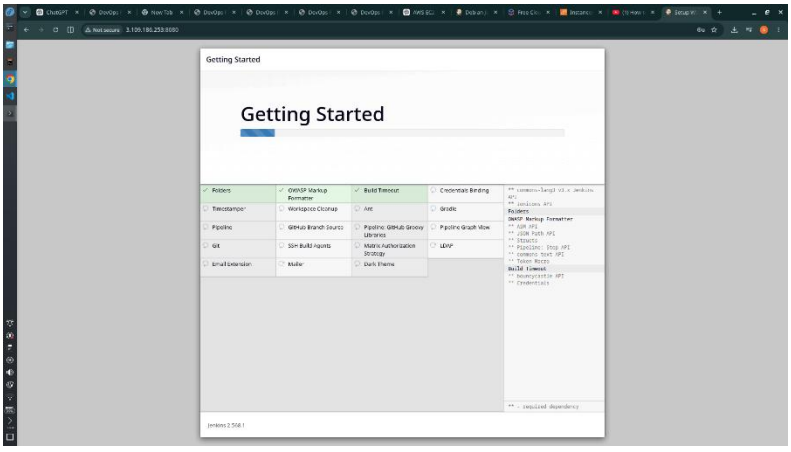
This final section captures the outcome of the entire deployment workflow described across Chapters 2 through 5 — the portfolio website, running inside a Docker container on an AWS EC2 instance, reachable by any visitor through the instance's public IP address. Reaching this point confirmed that every layer of the system, from the GitHub repository to the Nginx process inside the container, was working together correctly.

*[Insert Figure 6.11 Here]*

**Figure 6.11: Portfolio Website Successfully Accessed via the EC2 Public IP Address in a Browser**

This screenshot demonstrates the completed, end-to-end result of the project: the portfolio website loading correctly when accessed over the public internet. It serves as the visual conclusion of the deployment workflow documented throughout this report.



# CHAPTER 7

# CONCLUSION

## 7.1 Project Summary

This project, titled “Deployment of a Static Portfolio Website on AWS EC2 Using Docker and Nginx,” brought together two closely related areas of practical web engineering: the design and development of a responsive frontend portfolio website, and the deployment of that website to a cloud-based hosting environment using containerization. The work began with the construction of the website itself using HTML, CSS, and JavaScript, with careful attention given to page structure, visual presentation, responsiveness, and basic interactivity. The site was organized into six core sections — Home, About, Skills, Projects, Resume, and Contact — so that it could function not only as a technical exercise, but also as a meaningful personal portfolio artifact.

Once the website had been developed and verified in the local environment, the focus of the project shifted to version control, packaging, and deployment. Git was used to track changes to the source code, while GitHub served as the remote repository through which the code could be managed and transferred to the deployment server. An AWS EC2 instance running Ubuntu was then provisioned to act as the public host for the application. To support secure and functional access, the instance’s Security Group was configured to allow inbound traffic on port 22 for SSH and port 80 for HTTP. This stage established the infrastructure foundation required for hosting the application on the internet.

The deployment itself was completed through Docker and Nginx. Rather than installing a web server directly on the EC2 host system, the website was packaged into a Docker image based on the official Nginx image, allowing the static files to be served from within a containerized environment. This approach made the deployment more portable, reproducible, and isolated from host-level configuration differences. After the image had been built on the EC2 instance and run as a container with the appropriate port mapping, the website became accessible through the public IP address of the server. The successful display of the portfolio site in the browser confirmed that the full workflow — from development to cloud deployment — had been completed correctly.

In summary, the project achieved its primary objective of implementing and deploying a static portfolio website through a structured, containerized cloud deployment pipeline. It demonstrated that even a relatively simple static website can serve as an effective vehicle for learning and applying industry-relevant practices in version control, server provisioning, cloud networking, containerization, and web serving. The final result was not merely a working website, but a documented and repeatable deployment process that reflects both front-end development capability and foundational DevOps competence.

## 7.2 Learning Outcomes

A major outcome of this project was the practical understanding gained across multiple layers of modern web deployment. At the front-end level, the project strengthened knowledge of HTML, CSS, and JavaScript through the

design and implementation of a complete single-page portfolio website. HTML was used to define the semantic structure of the page and its sections, CSS was used to create a coherent visual design and responsive layout, and JavaScript was used to add interactive behaviour such as navigation support and user interface improvements. Through this process, the relationship between structure, styling, and client-side behaviour became clearer in a practical context rather than only in theory.

The project also provided valuable experience in Git and GitHub, both of which played an essential role in source code management. Using Git made it possible to track development changes systematically, while GitHub served as the remote location from which the project could be cloned onto the deployment server. This reinforced the importance of version control not only for collaboration, but also for deployment consistency, backup, and reproducibility. The use of a remote repository also mirrored a standard professional workflow, in which application code is maintained separately from the hosting environment.

At the infrastructure level, the project developed practical familiarity with AWS EC2 and the basic principles of cloud server administration. Launching the EC2 instance, selecting the operating system, managing access credentials, and configuring Security Groups provided hands-on exposure to how a cloud-hosted virtual machine is provisioned and secured. In addition, the use of SSH introduced the practical discipline of remote system access and command-line server management. Connecting to the instance, installing required software, and performing deployment tasks from the terminal helped build confidence in working with Linux-based cloud environments.

The project further strengthened understanding of Docker as a containerization platform and Nginx as a web server for static content. Writing the Dockerfile, selecting the Nginx base image, copying the portfolio files into the web root, building the image, and running the container illustrated the full lifecycle of container-based deployment. This showed clearly how containerization can package application files and runtime behaviour into a portable and repeatable unit. At the same time, using Nginx within the container demonstrated how static assets can be served efficiently without requiring a heavier application stack. Together, Docker and Nginx provided a practical introduction to deployment practices that are widely used in real-world hosting scenarios.

Beyond the individual technologies involved, the project also developed broader procedural and problem-solving skills. It required following a logical deployment sequence, verifying configuration at each stage, and diagnosing issues when infrastructure settings did not produce the expected result immediately. This helped reinforce an important lesson that successful deployment is not only about writing code, but also about understanding how code, servers, networking rules, and runtime environments interact. In this sense, the project served as a bridge between front-end development and operational deployment, giving a more complete understanding of how web applications are delivered to end users.

## 7.3 Future Enhancements

Although the project was successfully completed within its intended scope, several realistic enhancements could be introduced in the future to improve the deployment in terms of security, maintainability, scalability, and

professionalism. One of the most immediate improvements would be the addition of HTTPS through the use of SSL/TLS certificates. At present, the website is served over HTTP, which is sufficient for demonstrating the core deployment workflow but does not provide encrypted communication. Configuring HTTPS using a certificate solution such as Let's Encrypt would improve security, align the deployment with modern web standards, and provide a more professional presentation for public access.

A second improvement would be the integration of a custom domain name instead of relying solely on the EC2 instance's public IP address. Mapping the deployed site to a domain would make it easier to access, easier to remember, and more suitable for use as an actual professional portfolio. In a practical deployment, this step could be combined with DNS configuration and SSL certificate setup so that the website appears under a recognizable and secure web address. This would move the project closer to a production-style hosting arrangement.

Another important enhancement would be the introduction of a CI/CD pipeline so that updates to the portfolio could be deployed automatically whenever changes are pushed to the GitHub repository. At present, the deployment process is manual: code must be pulled onto the EC2 instance, the Docker image must be rebuilt, and the container must be restarted. A CI/CD workflow using tools such as GitHub Actions, Jenkins, or another automation platform could streamline this process by automatically building, testing, and redeploying the application after each approved code change. Such automation would reduce deployment effort and lower the risk of human error.

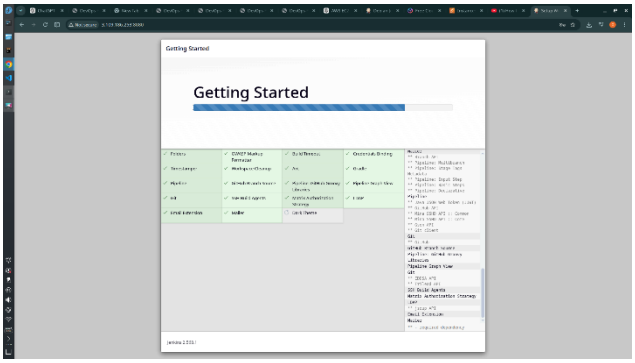
The project could also be strengthened through improved monitoring and observability. In its current form, the deployment confirms success primarily through direct browser access and manual inspection. In a more advanced environment, monitoring tools could be used to observe server health, container status, CPU and memory usage, uptime, and access logs. This would make it easier to detect failures, measure performance, and respond to operational issues more effectively. Log aggregation and alerting mechanisms could also be introduced so that the state of the deployment can be assessed continuously rather than only during manual testing.

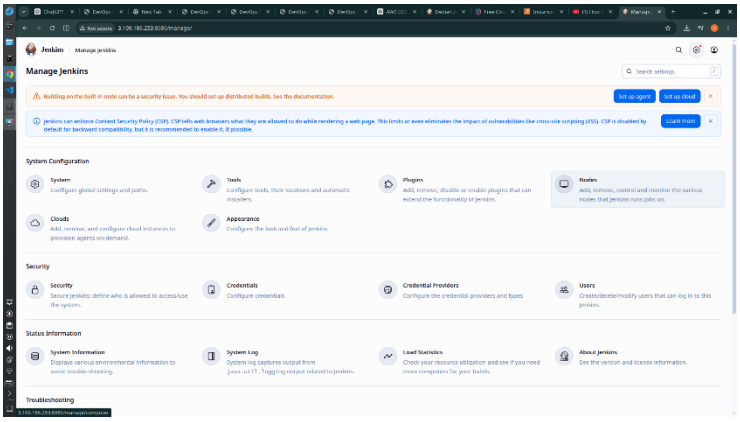
From a scalability perspective, the project could be extended through load balancing and eventually through container orchestration platforms such as Kubernetes. The current deployment uses a single EC2 instance and a single Docker container, which is appropriate for a small static site and for the learning objectives of this report. However, if the same deployment model were to be adapted for higher traffic or broader service requirements, a load balancer could distribute requests across multiple instances or containers. Kubernetes could then be introduced to manage container scheduling, scaling, and resilience automatically. While such technologies would go beyond the scope of the present project, they represent a logical progression from the foundation established here.

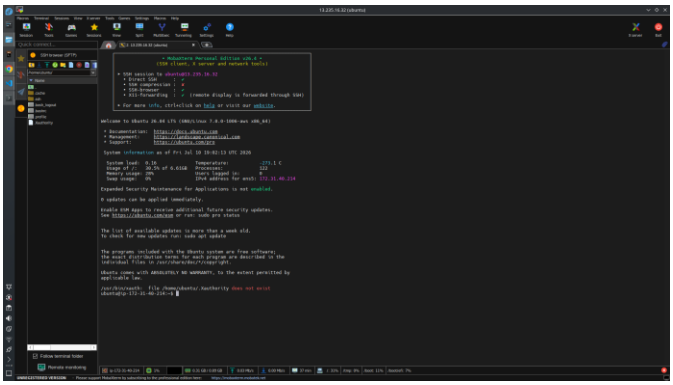
Finally, the portfolio website itself could be improved with additional user-facing features. The Projects section could be expanded with richer project descriptions, screenshots, live demo links, and categorized filtering. The Contact section could be enhanced through a working backend form handler instead of a static form, and the Resume section could include more dynamic presentation options. Accessibility improvements, performance optimization, animation refinement, and content personalization could also make the portfolio more effective as a professional showcase. These

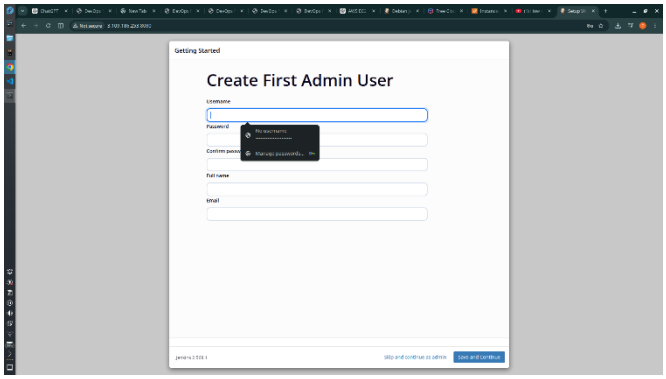
enhancements would not fundamentally alter the deployment model described in this report, but they would increase the practical value and presentation quality of the website being deployed.

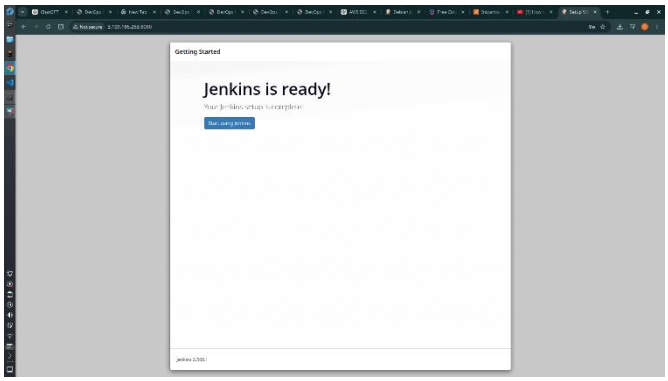
In conclusion, the present implementation successfully fulfilled its stated objectives while also creating a clear path for future development. The project established a solid foundation in front-end development, version control, cloud deployment, and containerized hosting, and it demonstrated how these areas can be integrated into a coherent workflow. The enhancements outlined above would build naturally on that foundation, allowing the project to evolve from a successful academic implementation into a more secure, automated, scalable, and feature-rich production-style deployment.





# CHAPTER 8

# REFERENCES

Throughout the design, development, and deployment of this project, a range of tools, platforms, and official documentation sources were consulted to ensure that each stage of the implementation — from the frontend code discussed in Chapter 3 to the AWS and Docker deployment steps discussed in Chapter 4 — followed established practice. This chapter lists the principal tools and reference sources used over the course of the project.

**Tools and Technologies**

| Tool / Platform | Purpose |
|---|---|
| HTML5, CSS3, JavaScript | Building the structure, styling, and interactivity of the portfolio website |
| Git | Local version control of the project source code |
| GitHub | Hosting the remote repository and providing the clone URL used on the EC2 instance |
| Visual Studio Code | Writing and editing the website's HTML, CSS, and JavaScript files |
| AWS EC2 | Provisioning the virtual server used to host the deployed website |
| Ubuntu Server | Operating system running on the EC2 instance |
| AWS Security Groups | Configuring inbound network access rules for the instance |
| Docker | Building and running the containerized Nginx and website image |
| Nginx | Serving the static website files over HTTP from inside the container |
| SSH / PuTTY-compatible terminal | Establishing a remote administrative connection to the EC2 instance |

**Documentation and Learning Resources**

| Source | Purpose |
|---|---|
| https://docs.aws.amazon.com | Official documentation for EC2, Security Groups, and related AWS services |
| https://docs.docker.com | Official documentation for Docker installation, Dockerfile syntax, and image/container commands |
| https://nginx.org/en/docs | Reference documentation for Nginx configuration and default behaviour |
| https://developer.mozilla.org | Reference for HTML, CSS, and JavaScript standards used in the frontend |
| https://git-scm.com/doc | Reference documentation for Git commands used in version control |
| https://docs.github.com | Guidance on creating and managing repositories on GitHub |
| https://ubuntu.com/server/docs | Reference documentation for Ubuntu Server package management and services |
| https://stackoverflow.com | Troubleshooting configuration and command-line issues encountered during deployment |

These resources were used strictly for reference and troubleshooting purposes; all code, configuration, and documentation presented in this report were written specifically for this project.